ClairSec

Adversarial FastAPI Security Platform

Final Presentation Guide & User Manual

Covers: setup, the 7-minute demo, GitHub import, offline / no-internet operation, every screen, the AI assistant, troubleshooting and panel Q&A.

> **One-line pitch:** ClairSec attacks your API the way a real attacker would — inside a sealed box — proves what it finds, writes the fix, and proves the fix works. No AI is ever allowed to grade its own work.

1. What ClairSec does

You give ClairSec a FastAPI project (a local folder or a GitHub URL). It runs a five-stage pipeline:

Stage	Plain words	What really happens
Builder	Reads your app and maps every door into it.	Static route extraction + OpenAPI from the running app, test users provisioned.
Attacker	Tries those doors like a real attacker — safely.	Sends real requests through a scope-locked client to the sandboxed app.
Evaluator	Double-checks every attack with hard rules.	Deterministic oracles (code, not AI) decide; reproduces each finding; CVSS computed by code.
Fixer	Writes a code change that closes the door.	Unified diff applied only to a copy of the project (original never touched).
Verifier	Tries the attack again and checks the app still works.	Builds the patched app in its own container; three proofs: exploit blocked, app still works, endpoint preserved.

> **The rule that makes it trustworthy:** AI models only *propose* tests and patches. Every verdict — confirmed, severity, score, "fix verified" — is computed by platform code from recorded requests and responses.

Key features to show

- **GitHub import** — paste a URL; safe tarball download (size, file-count, path and symlink checks), file tree, commit SHA, import notes, and **exposed-secret detection** (API keys/tokens/passwords found, shown *masked*).
- **Autopilot** — one click: import → sealed container → scan → fix → verify → report.
- **Live attack timeline**, agent cards and **attack-surface map**.
- **Security score** before → after verified fixes (e.g. 55 D → 100 A), with honest "limited coverage" notes.
- **"Why you can trust this finding"** — the evidence chain: AI proposal → request sent → oracle fired → reproduced → CVSS → fix verified.
- **Exploit replay** — re-send the same attack to the original app (works, HTTP 200) and the patched app (blocked, HTTP 403).
- **Three-proof fix review** — exploit blocked · app still works · endpoint preserved.
- **Reports** — PDF, HTML, Markdown, JSON and SARIF (GitHub code scanning format); executive page + technical detail.
- **Ask ClairSec** (Ctrl+J) — AI assistant on every screen, grounded in recorded data, with citations; it cannot change anything.
- **Themes** — light/dark, 5 colour presets, responsive layout, **Presenter mode (F11)**.

2. Setup (once)

1. **Docker Desktop** — start it and wait for "Engine running". If it hangs: quit Docker Desktop, run `wsl --shutdown` in PowerShell, start Docker Desktop again.
2. **MongoDB** — the `clairsec-mongo` container (already on this PC) starts with Docker. If MongoDB is not running the backend still works on **temporary in-memory storage** (badge: TEMPORARY STORAGE).
3. **Backend packages** (already installed in `venv`):

   ```
   cd D:\ClairSec\backend
   ..\venv\Scripts\python.exe -m pip install --require-hashes -r requirements.txt
   ```

4. **Flutter**: `cd D:\ClairSec\desktop` then `flutter pub get`.
5. **AI (optional)** — without a key ClairSec uses its offline assistant and offline AI automatically. For live AI put `LLM_PROVIDER=anthropic` and `LLM_API_KEY=...` in `backend\.env`, or install Ollama, `ollama pull <model>`, and set `LLM_PROVIDER=ollama` + `OLLAMA_MODEL=<model>`.

3. Start it (every time)

```
:: Terminal 1 — backend
cd D:\ClairSec\backend
..\venv\Scripts\python.exe -m uvicorn app.main:app --host 127.0.0.1 --port 8000

:: Terminal 2 — desktop app
cd D:\ClairSec\desktop
flutter run -d windows
```

Open **Dashboard**: the readiness banner shows Backend / MongoDB / Docker / AI / Git, each with its fix. The top bar shows the **mode badge**:

Badge	Meaning
LIVE	Docker + your AI key — full pipeline in containers with a real model.
LIVE · LOCAL AI	Docker + Ollama model on this PC.
LIVE · OFFLINE AI	Docker, no AI key — containers + built-in offline AI.
DEMO APP	No Docker — scans ClairSec's own bundled vulnerable app (real scan, ~15 s).
RECORDED RUN	A real earlier scan loaded from `demo/bundles` — no Docker, no internet needed.
TEMPORARY STORAGE	MongoDB not reachable — results kept in memory until the backend stops.

4. The 7-minute presentation

Before you start: press **F11** (Presenter mode — bigger text), maximise the window, pick Light theme if the room is bright.

Time	Do	Say
0:00	Dashboard	"AI writes more code every day; nobody has time to pen-test it. ClairSec attacks, proves, fixes and re-proves — and no AI grades its own homework."
0:30	Point at banner + mode badge	"It always tells you what it is running on. Nothing on screen is faked."
1:00	Projects → Import → GitHub URL (section 5) → tick "authorised" → Import	"It downloads a snapshot, never runs repo code on my laptop, refuses archives that try to escape — and look, it found committed secrets, masked."
2:00	Project card → **Run full check** (Autopilot); open the scan	"Builder maps the doors, Attacker tries them, Evaluator checks with hard rules, Fixer patches, Verifier re-attacks."
3:00	Security score card	"55, grade D, before fixes — 100, grade A, after verified fixes. Computed by code, never 'secure'."
3:30	Open the BOLA finding → "Why you can trust this finding"	"The AI only proposes. Every verdict after that is platform code."
4:30	**Replay attack** on Original, then Patched	"Same request, same judge. Before: leaked (200). After: blocked (403)."
5:30	View Fix → three proof cards	"Deleting the endpoint would 'block' the attack too — that's why the third proof exists."
6:00	**Ctrl+J** → Explain simply → "Is the fix safe?"	"It reads only recorded data, cites its sources and can't change anything."
6:30	Reports → Generate → **Open PDF**	"Executive page for managers, technical detail for developers, SARIF for GitHub."

With Docker the Autopilot run of a GitHub project takes about 1–5 minutes (container build). Start it before you begin talking, or use the Demo app (15 s) / a recorded run (instant) for the live part.

5. GitHub import — URLs that work

URL	Sub-folder (Advanced)	What you will see
`https://github.com/chiragk31/ClairSec`	`backend/tests/fixtures/vulnerable_fastapi_app`	**Best for the demo.** ClairSec's deliberately vulnerable shop API, imported like any user repo: file tree, commit SHA, exposed-secret notes, then (with Docker) container scan → 3 confirmed vulnerabilities (BOLA, mass assignment, debug config leak) → all fixed and verified → 55 → 100.
`https://github.com/chiragk31/ClairSec`	`backend/tests/fixtures/secure_fastapi_app`	The same API, correctly secured — "trust check": should find **0** confirmed problems.
`https://github.com/tiangolo/uvicorn-gunicorn-fastapi-docker`	(none)	A real third-party FastAPI repo (verified import: 20 files, entry point detected, editor-hook note). Small app, so expect few or no findings

		— good to show that ClairSec does not invent problems.

> **How:** Projects → Import → **GitHub URL** → paste the URL → (for ClairSec URLs) open the sub-folder field and paste the path → tick "I am authorised to test this code" → Import. Then press **Run full check** on the new project card. Import itself does not need Docker; scanning a GitHub project does.

6. Offline / no-internet / something-broke manual

> **Principle:** every fallback shows **real data** — either a real scan of ClairSec's own demo app, or a recording of a real earlier scan — and is always labelled with its badge. Say so if asked; it is a strength ("it never fakes results"), not a weakness.

Situation	What still works	What to click
No internet	Everything except GitHub import and cloud AI.	Projects → Import → **Demo app** → Scan Demo → Run full check.
Docker won't start	Demo app (runs on ClairSec's built-in runner), recorded runs, reports, assistant.	Demo app as above (badge DEMO APP).
MongoDB down	Everything; data is temporary.	Nothing — badge TEMPORARY STORAGE appears.
No AI key / no internet for AI	Offline AI in scans; offline assistant answers the suggested questions.	Nothing — assistant says "Offline assistant".
Live scan too slow / misbehaves	Recorded run of a real Demo scan.	Scans → **Watch a recorded run** → "Demo: Vulnerable FastAPI App" → Watch.
Backend won't start	The PDF/HTML report you generated earlier.	Open the saved PDF from `workspaces\reports`.
Laptop/projector disaster	This PDF and a saved report PDF.	Present from the PDFs.

Prepare before the presentation (10 minutes)

1. Run the Demo app once and the GitHub URL from section 5 once, so both are in scan history.
2. Generate a report for each and keep the PDFs open in a browser tab.
3. Check Scans → Watch a recorded run lists the two recordings.
4. Press F11, maximise the window, close other apps, disable notifications.

7. Screen-by-screen

Screen	What it shows
Dashboard	Readiness banner, key numbers, recent scans, top findings.
Projects	Import (local folder / GitHub URL / Demo app), project cards with import notes, exposed secrets (masked), isolation controls, Run full check.
Scans	History; Watch a recorded run. Scan detail: mode badge, agent cards, security score, live counters, attack timeline, attack-surface map, confirmed findings.
Vulnerabilities	Two lanes: Confirmed vs Candidates & suspicions. Finding detail: plain-words line, evidence chain, Replay attack (original vs patched), description, evidence, source location.
Fix Review	Diff + three proofs + variant attack + "No AI model voted on whether this fix works".
Reports	Generate, executive preview, Open PDF / HTML / Markdown / JSON / SARIF, Open folder, Compare with previous scan (New / Fixed / Still open / Came back).
Settings	Theme mode, colour preset, density, text size, reduce motion, Presenter mode.
Everywhere	Ctrl+K command palette, Ctrl+J Ask ClairSec, F11 presenter mode, status bar.

8. Ask ClairSec (AI assistant)

- Open with the ✦ button or **Ctrl+J**. It knows the screen you are on (scan, finding, fix…).
- Tap a suggested question or type your own; switch on **Explain simply** for non-technical answers.
- Answers show **citation chips** (the records used) and links you can click; it never changes anything.
- Without an AI key it is the **Offline assistant** (deterministic, from recorded data). With a key it writes the prose, but answers that cite nothing or claim an app "is secure" are withheld by platform code (this was caught live with a small local model following an injected instruction).

9. Troubleshooting

Symptom	Fix
Docker red / commands hang	Quit Docker Desktop → `wsl --shutdown` → start Docker Desktop → wait for "Engine running".
Backend unreachable	Start it (section 3). Port 8000 busy? Close the other process.
Build fails "Windows-only package"	Remove `pywin32` etc. from the project's requirements.
Build timeout	Set `DOCKER_BUILD_TIMEOUT=900` in `backend\.env`.
GitHub import fails	Check internet; check the URL and sub-folder; use Demo app.
Window too wide	Maximise it (double-click the title bar).
Anything else on stage	Demo app or Watch a recorded run (section 6).

10. Questions the panel may ask

"Isn't this just ChatGPT finding bugs?" No. The LLM only proposes tests and patches; oracles (code) decide. Show the evidence chain on a finding.

"How do you know the fix didn't break the app?" Three proofs: the exploit is re-run and blocked, the app's checks still pass (its own tests, or read-only checks generated from its OpenAPI schema and compared to before the fix), and the route still exists. Otherwise the fix is marked regressed or unverified, never "fixed".

"Is it safe to scan a random GitHub repo?" It is downloaded as a tarball with size, file-count, path and symlink checks, and runs only inside a hardened container (no capabilities, read-only filesystem, internal network). Only ClairSec's own demo apps ever run directly.

"What did you find in secrets?" Keys committed to the repo are detected and shown masked — we never display a working key.

"What is novel?" Verified-fix proof (blocked + still works + endpoint kept), an injection-resistant grounded assistant, and a fallback chain that never fakes data.

11. Evidence it works (measured)

- Backend test suite with Docker running: **387 passed** (including 15 container integration tests).
- Desktop: **114 widget tests passed**, analyzer clean; Windows app builds and was driven by an automated click-through against the live backend (every screen, a recorded run, the assistant).
- Live Docker run on a user project: isolation → scan → 3 fixes verified in patched containers → report; score 55 → 100; replay: original HTTP 200 (attack worked), patched HTTP 403 (blocked).
- Real GitHub import of a third-party repo; secret scanner finds committed keys and never stores their values.

Source of truth for developers: `docs/v2/` (PLAN, FEATURES, UI_SYSTEM, DEMO_SCRIPT, MANUAL_STEPS) and `docs/MEMORY.md`.